DOKUMENTASI TEKNIS

Alur Aplikasi dan Penjelasan Kode

Panduan membaca kode Similarity System untuk programmer junior: dari login, upload tugas, analisis kemiripan dan deteksi AI, sampai Lab Pengujian. Setiap langkah ditunjukkan file dan fungsinya.

Sistem	Similarity System, Laravel + FastAPI
Pembaca	Programmer yang paham dasar PHP/Python dan web
Untuk	Proyek Akhir D4 Rekayasa Perangkat Lunak
Versi	Oktober 2026

Daftar Isi

1. Kenalan Dulu dengan Sistemnya 4

1.1 Dua aplikasi yang bekerja sama 4

1.2 Istilah yang sering muncul 4

2. Menjalankan Sistem 5

2.1 Tiga container 5

2.2 Akun untuk mencoba 5

2.3 Peta folder 6

3. Gambaran Alur dari Awal sampai Akhir 6

4. Alur Login dan Hak Akses 7

4.1 Langkah login 7

4.2 Dua lapis penjaga 7

5. Alur Tugas dan Pengumpulan 8

5.1 Dosen menyiapkan tugas 8

5.2 Mahasiswa mengunggah tugas 8

6. Alur Inti: Menjalankan Analisis 9

6.1 Kenapa harus asinkron? 9

6.2 Langkah demi langkah di sisi Laravel 9

6.3 Keamanan antar aplikasi 10

7. Di Dalam FastAPI 11

7.1 Saat FastAPI menyala 11

7.2 Menerima pekerjaan 11

7.3 Langkah 0: membaca isi berkas 11

7.4 Langkah 1: Tahap 1, menghitung kemiripan 12

7.5 Langkah 2: Tahap 2, AI memeriksa pasangan yang dicurigai 13

7.6 Langkah 3: deteksi teks AI per dokumen 13

7.7 llm.py: cara bertanya ke AI 14

7.8 Mengirim hasil ke Laravel 14

8. Hasil Diterima, Ditampilkan, dan Dievaluasi 14

8.1 Laravel menerima webhook 14

8.2 Halaman hasil 15

8.3 Validasi dosen 15

8.4 Halaman evaluasi 15

9. Lab Pengujian (Role Peneliti) 16
9.1 Menyusun dataset 16
9.2 Menjalankan eksperimen 16
9.3 Menghitung hasil eksperimen 17
9.4 Uji cepat 18
10. Data yang Disimpan di Database 18
10.1 Istilah dasar 18
10.2 Asal data 18
10.3 Peta hubungan antar tabel 19
10.4 Akun dan perkuliahan 19
10.5 Hasil analisis kemiripan dan AI 21
10.6 Validasi dosen 24
10.7 Lab Pengujian 24
10.8 Tabel bawaan Laravel 27
10.9 Contoh perjalanan data 27
11. Kalau Ingin Mengubah Sesuatu 27

1. Kenalan Dulu dengan Sistemnya

Sistem ini membantu dosen mengecek dua hal pada tugas mahasiswa:

- **Kemiripan antar tugas**: apakah ada mahasiswa yang menyalin atau memparafrase tugas temannya.
- **Teks buatan AI**: apakah sebuah tugas kemungkinan ditulis oleh ChatGPT, Claude, dan sejenisnya.

1.1 Dua aplikasi yang bekerja sama

Sistem ini sebenarnya terdiri dari **dua aplikasi terpisah**. Bayangkan sebuah restoran:

Aplikasi	Perumpamaan	Tugasnya
Laravel (PHP)	Pelayan dan kasir	Semua yang dilihat pengguna: halaman login, mata kuliah, tugas, upload berkas, tombol "Jalankan analisis", halaman hasil, Lab Pengujian. Menyimpan semua data di MySQL.
FastAPI (Python)	Dapur	"Memasak" analisis: membaca isi PDF/DOCX, menghitung kemiripan, bertanya ke AI (Claude). Tidak punya database dan tidak mengurus login.

Pelayan tidak memasak, dan juru masak tidak melayani tamu. Begitu juga di sini: **Laravel tidak pernah menghitung kemiripan**, dan **FastAPI tidak pernah menampilkan halaman**. Aturan ini sengaja dijaga supaya tiap bagian mudah dirawat dan bisa diganti tanpa mengganggu yang lain.

1.2 Istilah yang sering muncul

Kalau sudah paham istilah di bawah, lewati saja bagian ini.

Istilah	Artinya di proyek ini
Route	Daftar alamat URL dan fungsi mana yang dijalankan. Contoh: *POST /login* → fungsi login.
Controller	Fungsi yang menerima request dari browser, lalu memanggil service, lalu mengembalikan halaman.
Service	Tempat logika utama ("otak"). Controller dibuat tipis, pekerjaan beratnya ada di service.
Model	Kelas PHP yang mewakili satu tabel database. Contoh: model *Submission* = tabel *submissions*.
Migration	File yang membuat atau mengubah struktur tabel database.
Middleware	Penjaga pintu yang dicek sebelum controller jalan. Contoh: "harus login", "harus dosen".
Policy	Aturan izin per data. Contoh: dosen hanya boleh mengubah mata kuliah miliknya sendiri.
Blade / view	File tampilan HTML Laravel (berakhiran *.blade.php*).
Endpoint	Alamat API yang bisa dipanggil program lain, misalnya */api/v1/analyze-similarity* di FastAPI.
Webhook / callback	Kebalikan dari biasanya: FastAPI yang *memanggil* Laravel untuk menyerahkan hasil.
Background task	Pekerjaan yang dijalankan di belakang layar, sehingga pemanggil tidak perlu menunggu.

Istilah	Artinya di proyek ini
Embedding	Mengubah kalimat menjadi deretan angka (vektor). Kalimat yang maknanya mirip menghasilkan angka yang mirip.
Cosine similarity	Cara mengukur kemiripan dua vektor. Hasilnya 0 sampai 1; makin dekat ke 1, makin mirip.
LLM	Large Language Model, yaitu AI seperti Claude atau Gemini yang bisa membaca dan menilai teks.
Docker container	"Kotak" berisi satu aplikasi lengkap dengan semua kebutuhannya, sehingga jalan sama di komputer mana pun.

2. Menjalankan Sistem

2.1 Tiga container

File `docker-compose.yml` menyalakan tiga container sekaligus:

Container	Nama	Alamat dari komputer	Isi
db	campus_db	localhost:3306	Database MySQL 8
fastapi	campus_fastapi	localhost:8000	Mesin analisis (Python)
laravel	campus_laravel	**http://localhost:8081**	Aplikasi web yang dibuka di browser

Di dalam Docker, ketiganya saling memanggil dengan **nama container**, bukan *localhost*. Laravel memanggil *http://fastapi:8000*, dan FastAPI memanggil balik *http://laravel/...*.

```
docker compose up -d            # nyalakan semua
docker compose logs -f fastapi  # lihat log FastAPI (berguna saat analisis jalan)
docker exec campus_laravel php artisan test   # jalankan semua test
```

2.2 Akun untuk mencoba

Role	Email	Password	Bisa apa
Admin	admin@kampus.ac.id	password	Melihat ringkasan sistem
Dosen	dosen@kampus.ac.id	password	Mengelola mata kuliah, tugas, analisis, validasi
Mahasiswa	andi@student.kampus.ac.id	password	Mengumpulkan tugas
Peneliti	peneliti@kampus.ac.id	password	Lab Pengujian

2.3 Peta folder

```
similarity-system/
├── docker-compose.yml        cara menyalakan 3 container
├── .env                      kunci rahasia (API key) - jangan di-commit
├── fastapi-service/          aplikasi Python (dapur)
│   ├── main.py               endpoint & alur analisis
│   ├── extraction.py         baca teks PDF/DOCX + cek dokumen bergambar
│   ├── multilayer.py         hitung kemiripan (Tahap 1)
│   └── llm.py                tanya ke Claude/Gemini (Tahap 2 & deteksi AI)
└── laravel-app/              aplikasi web (pelayan & kasir)
    ├── routes/web.php        alamat halaman
    ├── routes/api.php        alamat webhook dari FastAPI
    ├── app/Http/Controllers  controller per role (Dosen, Mahasiswa, Peneliti, Api)
    ├── app/Http/Middleware   penjaga: cek role, cek kunci webhook
    ├── app/Http/Requests     aturan validasi form
    ├── app/Policies          aturan izin per data
    ├── app/Services          LOGIKA UTAMA ada di sini
    ├── app/Repositories      penyimpanan hasil analisis
    ├── app/Models            satu kelas per tabel
    ├── resources/views       tampilan (Blade)
    ├── database/migrations   struktur tabel
    └── tests/Feature         test otomatis
```

Tips membaca kode
Saat membaca kode Laravel, biasakan urutan ini: **route** → **controller** → **service** → **model**. Cari URL-nya di *routes/web.php*, lihat controller mana yang dipanggil, lalu ikuti ke service-nya.

3. Gambaran Alur dari Awal sampai Akhir

Sebelum masuk ke detail, ini urutan besar yang terjadi di sistem:

1. **Login.** Pengguna masuk dan diarahkan ke dashboard sesuai role-nya.
2. **Dosen menyiapkan** mata kuliah, mendaftarkan mahasiswa, dan membuat tugas dengan tenggat.
3. **Mahasiswa mengunggah** tugas (PDF/DOCX). Bisa diganti atau dihapus selama tugas masih dibuka.
4. **Tugas ditutup**, karena tenggat lewat atau ditutup manual oleh dosen.
5. **Dosen menekan "Jalankan analisis".** Laravel mengirim daftar berkas ke FastAPI.
6. **FastAPI menganalisis** di belakang layar: hitung kemiripan semua pasangan, tanya AI untuk pasangan yang mencurigakan, dan cek apakah tiap tugas ditulis AI.
7. **FastAPI mengirim hasil** ke Laravel lewat webhook. Laravel menyimpannya.
8. **Dosen melihat hasil** dan menandai mana yang benar plagiat (validasi).
9. **Halaman Evaluasi** menghitung seberapa akurat sistem dibanding penilaian dosen.

Selain itu ada **Lab Pengujian** khusus role peneliti, untuk menguji dan menyetel metode memakai dokumen yang labelnya sudah diketahui (bagian 9).

4. Alur Login dan Hak Akses

4.1 Langkah login

Langkah 1 — Membuka aplikasi

Alamat */* dialihkan ke */dashboard*. Karena belum login, middleware *auth* mengalihkan lagi ke */login*.

Langkah 2 — Mengirim email dan password

Form dikirim ke *POST /login*, yang ditangani `AuthenticatedSessionController::store()`.

Sebelum itu, `LoginRequest` memvalidasi isian, mengecek password, dan **membatasi percobaan**: kalau salah terlalu sering, akun dikunci sementara (mencegah tebak-tebakan password).

Langkah 3 — Diarahkan ke dashboard

Setelah berhasil, sesi dibuat ulang (*session()->regenerate()*, demi keamanan) lalu pengguna dikirim ke */dashboard*.

`DashboardController::index()` memanggil `User::dashboardRoute()` untuk memilih dashboard yang sesuai role.

```
// app/Models/User.php
public function dashboardRoute(): string
{
    return match ($this->role) {
        'admin'     => 'admin.dashboard',
        'dosen'     => 'dosen.dashboard',
        'mahasiswa' => 'mahasiswa.dashboard',
        'peneliti'  => 'peneliti.dashboard',
    };
}
```

4.2 Dua lapis penjaga

Ada dua pemeriksaan izin yang berbeda, dan keduanya penting:

Penjaga	Pertanyaannya	File
Middleware role	"Apakah kamu dosen?" Kalau tidak → error 403.	`app/Http/Middleware/EnsureUserHasRole.php`
Policy	"Apakah mata kuliah ini milikmu?" Dosen A tidak boleh mengubah tugas dosen B.	`app/Policies/CoursePolicy.php` `app/Policies/AssignmentPolicy.php`

Middleware dipasang di route (*->middleware('role:dosen')*). Policy dipanggil di dalam controller dengan *Gate::authorize('manage', $assignment)*.

5. Alur Tugas dan Pengumpulan

5.1 Dosen menyiapkan tugas

Aksi dosen	Controller	Catatan
Membuat mata kuliah	`Dosen/CourseController`	Validasi di `CourseRequest`
Mendaftarkan mahasiswa	`Dosen/CourseStudentController`	Memilih dari daftar akun mahasiswa. Disimpan di tabel *course_user*.
Membuat tugas	`Dosen/AssignmentController::store()`	Validasi di `AssignmentRequest`
Menutup / membuka lagi	`AssignmentController::close()` / `reopen()`	Mengisi atau mengosongkan kolom *closed_at*

Kapan sebuah tugas dianggap masih dibuka? Jawabannya ada di model `Assignment`:

```
public function isOpen(): bool
{
    // dibuka = belum ditutup manual DAN tenggat belum lewat
    return !$this->isClosedManually() && !$this->isPastDue();
}
```

5.2 Mahasiswa mengunggah tugas

Langkah 1 — Membuka halaman tugas

`Mahasiswa/SubmissionController::show()` menampilkan detail tugas dan status pengumpulan mahasiswa itu.

Langkah 2 — Mengunggah berkas

Form dikirim ke `SubmissionController::store()`. Sebelumnya `SubmissionFileRequest` memastikan berkas berformat PDF atau DOCX, **isinya** juga benar-benar PDF/DOCX (bukan file lain yang diganti namanya), dan ukurannya maksimal 10 MB.

Policy *submit* memastikan mahasiswa terdaftar di mata kuliah itu dan tugas masih dibuka.

Langkah 3 — Menyimpan berkas

`SubmissionService::store()` menyimpan berkas ke *storage/app/public/submissions/{id_tugas}/* dengan nama acak (UUID), misalnya *9f1c...e2.pdf*. Nama acak membuat berkas tidak bisa ditebak orang lain. Nama asli berkas disimpan di database untuk ditampilkan.

Ekstensi (.pdf / .docx) tetap dipertahankan, karena nanti FastAPI menentukan cara membaca berkas dari ekstensinya.

Mengganti dan menghapus juga lewat `SubmissionService`:

- `replace()` — simpan berkas baru, lalu **hapus hasil analisis dan label lama** milik pengumpulan itu (karena isinya sudah berubah), baru hapus berkas lama.
- `delete()` — hapus baris database (hasil analisisnya ikut terhapus otomatis lewat *foreign key cascade*), lalu hapus berkasnya.

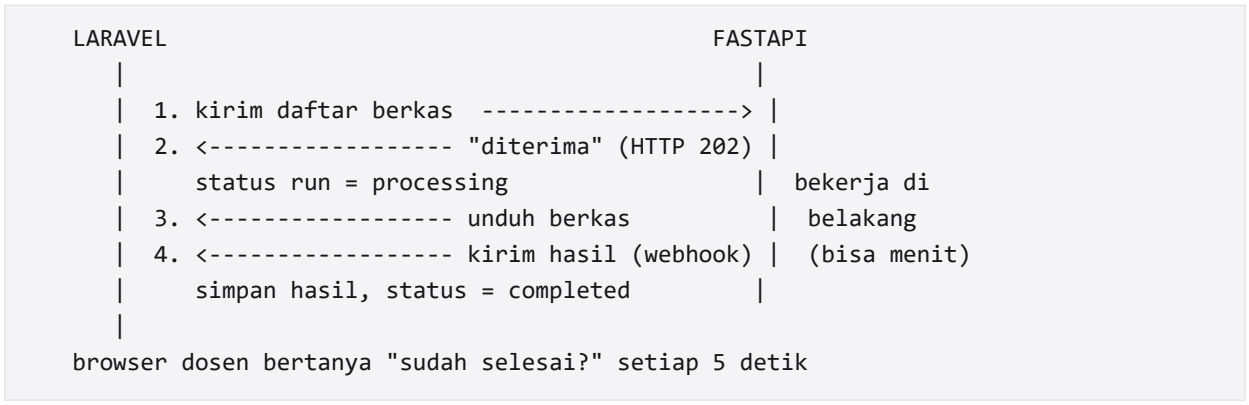
6. Alur Inti: Menjalankan Analisis

Ini bagian terpenting. Kita ikuti apa yang terjadi sejak dosen menekan tombol sampai hasil muncul.

6.1 Kenapa harus asinkron?

Menganalisis 30 tugas berarti membandingkan 435 pasangan, ditambah puluhan pertanyaan ke AI. Ini bisa memakan beberapa menit. Kalau Laravel menunggu sampai selesai, browser dosen akan *timeout*.

Solusinya seperti memesan makanan dengan nomor antrean: FastAPI langsung menjawab "pesanan diterima", bekerja di belakang, lalu **memanggil balik Laravel** ketika selesai.



6.2 Langkah demi langkah di sisi Laravel

Langkah 1 — Halaman tugas menyiapkan tombol

`Dosen/AssignmentController::show()` memanggil `AnalysisService` untuk dua hal:

• `partitionSubmissions()`: memisahkan pengumpulan yang berkasnya **benar-benar ada** di penyimpanan dan yang hilang.

• `blockingReason()`: mencari alasan tombol harus dimatikan, yaitu tugas masih dibuka, berkas yang ada kurang dari 2, atau masih ada analisis yang berjalan.

Langkah 2 — Dosen menekan tombol

Request masuk ke `Dosen/AnalysisController::store()`. Controller mengecek izin (policy), memvalidasi **ambang** (0,50 sampai 0,95; bawaan 0,70), lalu memanggil `AnalysisService::start()`.

Langkah 3 — Laravel mengirim pekerjaan ke FastAPI

`AnalysisService::start()` membuat satu baris di tabel *analysis_runs* (status *pending*), lalu mengirim JSON seperti ini:

```
POST http://fastapi:8000/api/v1/analyze-similarity
Header: X-API-Key: <kunci rahasia>
{
  "assignment_id": 12,
  "analysis_run_id": 34,
  "callback_url": "http://laravel/api/webhooks/similarity-result",
  "threshold": 0.70,
  "documents": [
    {"submission_id": 101, "student_name": "Budi",
     "file_url": "http://laravel/storage/submissions/12/9f1c...e2.pdf"},
    ...
  ]
}
```

Perhatikan: Laravel **hanya mengirim alamat berkas**, bukan isinya. FastAPI yang mengunduh dan membaca sendiri.

Kalau FastAPI mati atau menolak, run langsung ditandai *failed* lewat `markFailed()` dan dosen melihat pesan error. Kalau diterima, status berubah menjadi *processing*.

Langkah 4 — Browser menunggu

Di `resources/views/dosen/assignments/show.blade.php` ada JavaScript kecil yang setiap 5 detik memanggil `AnalysisController::status()`. Begitu statusnya bukan lagi *pending/processing*, halaman dimuat ulang dan hasil tampil.

Pengaman: kalau tidak ada kabar dari FastAPI selama 30 menit, `expireStaleRuns()` menandai run itu gagal supaya dosen bisa mencoba lagi.

6.3 Keamanan antar aplikasi

Webhook adalah alamat publik. Tanpa pengaman, siapa pun bisa mengirim "hasil palsu". Karena itu setiap panggilan membawa header **X-API-Key**:

Arah	Nama kunci di .env	Dicek oleh
Laravel → FastAPI	ANALYSIS_ENGINE_KEY	Middleware *require_engine_key* di `main.py`
FastAPI → Laravel	ANALYSIS_WEBHOOK_KEY	`VerifyAnalysisWebhookKey.php`

Kunci yang salah ditolak dengan HTTP 401. Kunci dibandingkan dengan *hash_equals* (PHP) dan *hmac.compare_digest* (Python), yaitu cara membandingkan yang aman dari serangan menebak lewat lama waktu proses.

7. Di Dalam FastAPI

FastAPI terdiri dari empat file Python:

File	Isinya
`main.py`	Endpoint, urutan kerja analisis, kalimat perintah (prompt) untuk AI, dan pengiriman hasil ke Laravel.
`extraction.py`	Membaca teks dari PDF/DOCX/TXT, menghitung kata/halaman/gambar, dan memberi peringatan dokumen bergambar.
`multilayer.py`	Perhitungan kemiripan berlapis (Tahap 1).
`llm.py`	Cara bertanya ke Claude atau Gemini.

7.1 Saat FastAPI menyala

- Model embedding **paraphrase-multilingual-MiniLM-L12-v2** dimuat **sekali** ke memori. Model ini mengubah kalimat (termasuk bahasa Indonesia) menjadi 384 angka.
- Parameter bawaan dibaca dari environment: panjang passage 60 kata, ambang kalimat 0,85, bobot 0,2 / 0,4 / 0,4, dan 5 passage bukti.

7.2 Menerima pekerjaan

Fungsi `analyze_similarity()` menerima request dari Laravel:

1. Isi JSON dicek otomatis oleh **Pydantic** (kelas *AnalysisRequest*, *DocumentItem*, *RunConfig*). Kalau formatnya salah, langsung ditolak.
2. `effective_params()` menggabungkan parameter kiriman Laravel (kalau ada, dari Lab) dengan nilai bawaan.
3. Pekerjaan dimasukkan ke **BackgroundTasks**, lalu FastAPI langsung menjawab **202 Accepted**.

Pekerjaan sebenarnya dijalankan `process_similarity_task()`. Fungsi ini memakai *try/except* sehingga apa pun yang terjadi, berhasil atau gagal, **hasilnya selalu dikirim ke Laravel**. Dengan begitu Laravel tidak menunggu selamanya.

7.3 Langkah 0: membaca isi berkas

`download_and_extract_text()` mengunduh berkas dari Laravel, lalu `extraction.py` mengambil teksnya:

- **.pdf** dibaca dengan pustaka *pypdf*, halaman demi halaman.
- **.docx** dibaca dengan pustaka *python-docx* sesuai urutan dokumen: paragraf, **isi tabel**, dan text box. Header/footer dilewati karena biasanya berisi nama atau judul yang sama untuk semua mahasiswa.
- **.txt** dibaca langsung.

Bagaimana dengan gambar?

Gambar **tidak dibaca** (tidak ada OCR). Screenshot berisi tulisan, diagram, atau halaman hasil scan tidak ikut dibandingkan. Supaya dosen tahu, setiap dokumen dihitung jumlah kata, halaman, dan gambarnya, lalu `text_warning()` memberi peringatan jika:

- tidak ada teks sama sekali (dokumen ini tidak ikut dianalisis);
- kurang dari 50 kata per halaman PDF;
- kurang dari 100 kata per gambar;
- kurang dari 50 kata secara keseluruhan.

Peringatan ini dikirim ke Laravel di *metrics.document_checks* dan tampil sebagai panel **"Perlu diperiksa manual"** di halaman hasil (komponen `components/document-checks.blade.php`). Jika tersisa kurang dari 2 dokumen yang terbaca, analisis dihentikan dengan pesan yang jelas.

7.4 Langkah 1: Tahap 1, menghitung kemiripan

Masalahnya

Model embedding hanya mampu membaca sekitar **128 token** (± 90 kata) sekali jalan. Kalau satu esai 1.500 kata dimasukkan utuh, sisanya diabaikan. Karena itu dokumen **dipecah dulu**.

Memecah dokumen — multilayer.encode_document()

1. **Rapikan teks** (`normalize_text()`): satukan spasi berlebih dan sambung kata yang terpotong di PDF, misalnya "iden- tified" menjadi "identified".
2. **Pecah per kalimat** (`split_sentences()`): dipotong setelah tanda . ! ? yang diikuti huruf kapital. Potongan di bawah 4 kata (judul, nomor halaman) digabung ke kalimat berikutnya.
3. **Susun passage** (`build_passages()`): kalimat-kalimat berurutan digabung sampai maksimal 60 kata. Satu passage kira-kira satu paragraf pendek.
4. **Ubah ke angka**: semua kalimat dan passage di-encode menjadi vektor.
5. **Vektor dokumen** = rata-rata semua vektor passage.

```
Dokumen (1.200 kata)
  ├─ 60 kalimat   → 60 vektor kalimat
  ├─ 20 passage   → 20 vektor passage
  └─ 1 dokumen    → 1 vektor (rata-rata 20 vektor passage)
```

Membandingkan dua dokumen — multilayer.pair_scores()

Untuk N dokumen ada N×(N−1)/2 pasangan (10 dokumen → 45 pasangan). Setiap pasangan diberi tiga skor:

Skor	Cara hitung	Artinya
D dokumen	Cosine vektor dokumen A dengan vektor dokumen B.	Seberapa mirip **topik** secara keseluruhan.
P passage	Setiap passage A dicarikan passage B yang paling mirip, lalu nilainya dirata-rata. Hal yang sama dilakukan dari arah B ke A, lalu kedua hasil dirata-rata.	Seberapa banyak **paragraf** yang punya pasangan makna.

Skor	Cara hitung	Artinya
K kalimat	Setiap kalimat A dicarikan kalimat B yang paling mirip. Kalau nilainya ≥ 0,85, kalimat itu "punya kembaran". K = persentase kalimat yang punya kembaran, dihitung dua arah lalu dirata-rata.	Berapa persen **kalimat** yang disalin atau hampir sama.

Karena setiap kalimat dibandingkan dengan **semua** kalimat lawannya, mengacak urutan kalimat tidak bisa mengelabui sistem.

Skor gabungan — multilayer.combine()

```
gabungan = (0,2 × D + 0,4 × P + 0,4 × K) / (0,2 + 0,4 + 0,4)

contoh: D = 0,90  P = 0,80  K = 0,50
        gabungan = 0,18 + 0,32 + 0,20 = 0,70  → ≥ ambang 0,70 → DICURIGAI
```

Pasangan dengan skor gabungan di atas atau sama dengan ambang disebut **ter-flag** (dicurigai).

Memilih bukti untuk AI

Dari perbandingan passage tadi, sistem mengambil **5 pasangan passage dengan nilai tertinggi**, dengan syarat satu passage tidak dipakai dua kali (agar bukti tidak berisi paragraf yang sama berulang). Lima pasangan ini nanti dibaca oleh AI.

7.5 Langkah 2: Tahap 2, AI memeriksa pasangan yang dicurigai

Hanya pasangan ter-flag yang dikirim ke AI lewat `analyze_similarity_with_llm()`. Pasangan lain tidak perlu, dan di sinilah penghematan biaya terjadi.

- **Yang dikirim ke AI:** 5 pasangan passage bukti, bukan dokumen utuh. Lebih hemat dan tetap mencakup bagian tengah/akhir dokumen.
- **Yang diminta:** apakah benar plagiat; kutipan bagian yang mirip dari A dan B; jenisnya *VERBATIM_COPY* (salin persis) atau *PARAPHRASED* (parafrase); rekomendasi *PERIKSA_MANUAL* atau *AMAN*. AI juga diminta mengabaikan kemiripan yang hanya karena topik tugasnya sama.
- **Kalau AI gagal menjawab:** hasil ditandai *llm_error* dan direkomendasikan *PERIKSA_MANUAL*, bukan dianggap aman.

7.6 Langkah 3: deteksi teks AI per dokumen

`detect_ai_generated_text()` dijalankan untuk setiap dokumen:

- Yang dikirim: cuplikan maksimal 3.000 karakter yang diambil **merata dari awal, tengah, dan akhir** dokumen (`sample_excerpt()`).
- AI menjawab *ai_probability* (0 sampai 1), ringkasan, dan contoh kalimat yang khas AI.
- Keputusan akhir dihitung ulang oleh kode dari probabilitas (`verdict_from_probability()`): ≥ 0,70 kemungkinan AI, ≥ 0,35 campuran, selain itu tulisan manusia.
- Kalau gagal, hasilnya *ERROR*. Kegagalan tidak boleh disamarkan sebagai "tulisan manusia".

7.7 llm.py: cara bertanya ke AI

Semua pertanyaan ke AI lewat satu fungsi, `call_llm_json()`. Fungsi ini memilih penyedia secara otomatis:

Penyedia	Dipakai kapan	Cara kerjanya
Claude	Jika ANTHROPIC_API_KEY diisi	Model claude-opus-5-5 dengan effort "low" (pintar tapi hemat token). Jawaban **dipaksa mengikuti format JSON** (structured outputs), jadi selalu bisa dibaca kode. Kalau terkena batas kuota atau server sibuk, SDK mencoba ulang otomatis hingga 4 kali.
Gemini	Jika tidak ada kunci Claude	Mencoba beberapa model bergantian. Jawaban dibersihkan dulu dari pembungkus ```json sebelum dibaca.

Setiap panggilan dihitung: jumlah panggilan, yang gagal, dan jumlah token masuk/keluar. Angka ini dikirim ke Laravel sebagai catatan biaya.

7.8 Mengirim hasil ke Laravel

`send_callback()` mengirim JSON ke *callback_url* dengan kunci webhook. Isinya kira-kira:

```
{
  "status": "completed",
  "analysis_run_id": 34,
  "results": [                          ← satu item per pasangan
    {"doc_a": {...}, "doc_b": {...},
     "similarity_score": 0.82,
     "layer_scores": {"document": 0.91, "passage": 0.84, "sentence": 0.76},
     "is_flagged": true,
     "llm_analysis": {"verdict": ..., "matched_segments": [...]}}
  ],
  "ai_detections": [ {"submission_id": 101, "ai_probability": 0.12, ...} ],
  "metrics": { waktu tiap tahap, jumlah panggilan AI, token, ... }
}
```

8. Hasil Diterima, Ditampilkan, dan Dievaluasi

8.1 Laravel menerima webhook

Alamat webhook ada di `routes/api.php`. Urutannya:

1. `VerifyAnalysisWebhookKey` mengecek kunci. Salah → 401, belum diatur → 503.
2. `Api/WebhookController` mencatat ke log lalu meneruskan ke service.
3. `SimilarityService::handleWebhookCallback()` mencocokkan hasil dengan run (lewat *analysis_run_id*) dan memastikan run itu memang milik tugas tersebut. Jika statusnya gagal, pesan error disimpan. Jika berhasil, semua penyimpanan dilakukan dalam **satu transaksi database**: hapus hasil lama, simpan hasil baru, isi metrik, status *completed*. Kalau satu bagian gagal, semuanya dibatalkan.

4. `SimilarityRepository` melakukan penyimpanan sambil **membersihkan data**: pasangan yang bukan milik tugas ini diabaikan, nilai yang aneh diganti nilai aman, dan hasil AI yang rusak ditandai ERROR.

Kenapa ada Repository? Supaya kode "kapan menyimpan" (service) terpisah dari "bagaimana menyimpan" (repository). Antarmukanya ada di *app/Repositories/Contracts* dan dihubungkan di *AppServiceProvider*.

8.2 Halaman hasil

`Dosen/AnalysisResultController` punya dua halaman:

- `index()` — ringkasan: daftar pasangan dari skor tertinggi, matriks kemiripan antar mahasiswa, dan hasil deteksi AI.
- `pair()` — detail satu pasangan: skor D/P/K, ringkasan dari AI, dan potongan teks yang mirip ditampilkan berdampingan.

Teks label di tampilan (misalnya LIKELY_AI menjadi "Kemungkinan AI") diatur di `app/Support/AnalysisLabels.php`.

8.3 Validasi dosen

Sistem tidak bisa menilai dirinya sendiri, jadi dosen menandai apakah hasilnya benar. `Dosen/GroundTruthController`:

- `storePair()` — label pasangan: plagiat atau bukan. Disimpan di tabel *pair_labels*. Id yang kecil selalu disimpan di kolom *low* dan yang besar di kolom *high*, supaya pasangan A–B dan B–A tidak tercatat dua kali.
- `storeAi()` — label dokumen: ditulis AI atau manusia. Disimpan di *ai_labels*.

Tombol label memakai komponen `components/label-toggle.blade.php` yang mengirim lewat *fetch*, jadi halaman tidak perlu dimuat ulang.

8.4 Halaman evaluasi

`EvaluationService` membandingkan tebakan sistem dengan label dosen:

- `labeledPairs()` — ambil pasangan yang sudah dilabeli. Tebakan **Tahap 1** = ter-flag. Tebakan **Tahap 1+2** = ter-flag dan AI tidak bilang AMAN.
- `confusion()` — menghitung TP, FP, FN, TN, lalu precision, recall, F1, dan akurasi. Fungsi ini juga dipakai Lab Pengujian.
- `summarize()` — ringkasan lengkap, termasuk mencoba ambang 0,50 sampai 0,95 untuk melihat ambang mana yang paling pas.

Singkatan	Artinya
TP	Sistem bilang plagiat, dan memang plagiat (benar)
FP	Sistem bilang plagiat, padahal bukan (salah tuduh)
FN	Sistem bilang aman, padahal plagiat (kelolosan)

Singkatan	Artinya
TN	Sistem bilang aman, dan memang aman (benar)

9. Lab Pengujian (Role Peneliti)

Lab dipakai untuk **menguji dan menyetel metode**, terpisah dari data kuliah (tabelnya berawalan *experiment_*). Dua perbedaan utamanya:

- **Label benar/salah otomatis.** Peneliti memberi kategori setiap dokumen (asli, salinan, parafrase, dan seterusnya), jadi tidak perlu dosen menilai satu per satu.
- **Parameter bisa diubah setiap percobaan**: ambang, bobot D/P/K, panjang passage, ambang kalimat, Tahap 2 ya/tidak, dan deteksi AI ya/tidak.

9.1 Menyusun dataset

File	Gunanya
`Peneliti/DatasetController`	Membuat, membuka, dan menghapus dataset. *storeSample()* membuat korpus contoh.
`Peneliti/DocumentController`	*store()* tambah satu dokumen, *import()* impor massal, *template()* unduh templat CSV, *show()*, *destroy()*.
`ExperimentService::addDocument()`	Menyimpan berkas atau teks ke *storage/app/public/experiments/{dataset}/*.
`ExperimentImportService`	Membaca CSV, mencocokkan dengan berkas yang diunggah, mengecek semua baris dulu, baru menyimpan semuanya sekaligus.
`ExperimentDocument`	Daftar kategori dokumen. Kategori turunan (salinan, parafrase manual, parafrase AI) wajib punya dokumen sumber.

Bagaimana label dibuat otomatis?

Setiap dokumen turunan menunjuk sumbernya. Sistem menelusuri rantai sumber itu sampai ke dokumen paling awal (`ExperimentEvaluator::rootOf()`). Dokumen-dokumen yang ujungnya sama dianggap satu "keluarga".

```
Esai 1 (asli)
 ├── Esai 1 - salinan          ┐
 ├── Esai 1 - parafrase        ├ satu keluarga → setiap pasangan di dalamnya = PLAGIAT
 │    └── salinan parafrase    ┘
Esai 2 (independen)             → dengan keluarga Esai 1 = BUKAN PLAGIAT
```

9.2 Menjalankan eksperimen

1. Peneliti mengisi form → `Peneliti/RunController::store()`.
2. `ExperimentService::startRun()` mencatat run di *experiment_runs* lalu mengirim ke **endpoint FastAPI yang sama** dengan analisis dosen, ditambah blok *config* berisi parameter percobaan.
3. FastAPI bekerja persis seperti bagian 7, tetapi memakai parameter dari *config*.

4. Hasil kembali lewat */api/webhooks/experiment-result* ke `ExperimentWebhookController` → `ExperimentService::handleCallback()`.
5. Yang disimpan adalah **skor mentah D, P, K** setiap pasangan (tabel *experiment_pair_scores*) dan probabilitas AI (tabel *experiment_ai_scores*).
6. Halaman run mengecek status tiap 4 detik, lalu memuat ulang saat selesai.

Kenapa skor mentah disimpan?
Karena D, P, K disimpan terpisah, **bobot dan ambang lain bisa langsung dihitung ulang di Laravel** tanpa memanggil FastAPI atau AI lagi.
Panjang passage dan ambang kalimat **tidak bisa** dihitung ulang begitu, karena keduanya mengubah cara D/P/K dihitung di FastAPI. Untuk mengganti keduanya, jalankan eksperimen baru.

9.3 Menghitung hasil eksperimen

Halaman hasil (`RunController::show()`) memakai fungsi-fungsi di `ExperimentEvaluator`:

Fungsi	Yang dihasilkan
`rows()`	Daftar semua pasangan beserta label otomatis, kategori, dan skor.
`evaluate()`	Hitung hasil untuk bobot dan ambang tertentu. Dipakai form "coba bobot & ambang lain".
`tierComparison()`	Bandingkan hasil Tahap 1 saja dengan Tahap 1+2.
`perCategory()`	Persentase salinan, parafrase manual, dan parafrase AI yang tertangkap, serta jumlah salah tuduh.
`sweep()`	F1 untuk ambang 0,30 sampai 0,95 (bahan grafik).
`ablation()`	Bandingkan: dokumen saja, passage saja, kalimat saja, bobot sama rata, bobot run, dan bobot hasil optimasi.
`weightGrid()`	Semua kombinasi bobot dengan langkah 0,1 yang jumlahnya 1 (ada 66 kombinasi).
`bestParams()`	Mencari bobot + ambang dengan F1 tertinggi. Kalau seri, pilih yang precision-nya lebih tinggi.
`crossValidate()`	5-fold cross-validation (lihat di bawah).
`aiEvaluation()`	Hasil deteksi AI per kategori dan percobaan ambang probabilitas 0,10 sampai 0,90.

Cross-validation dengan bahasa sederhana

Kalau bobot terbaik dicari *dan* dinilai memakai data yang sama, hasilnya terlalu bagus, seperti siswa yang mengerjakan ujian dengan soal yang sama persis dengan latihan. Maka data dibagi 5 kelompok (`folds()`, acakan tetap dengan seed 42):

```
putaran 1:  latih di kelompok 2,3,4,5  →  uji di kelompok 1
putaran 2:  latih di kelompok 1,3,4,5  →  uji di kelompok 2
...
putaran 5:  latih di kelompok 1,2,3,4  →  uji di kelompok 5

hasil akhir = rata-rata F1 dari 5 putaran ± simpangan bakunya
```

9.4 Uji cepat

`Peneliti/CompareController` mengirim dua teks ke endpoint FastAPI */api/v1/compare-texts*. Berbeda dengan analisis biasa, endpoint ini **langsung menjawab** (tidak lewat webhook) dan hasilnya tidak disimpan. Parameternya selalu nilai bawaan.

10. Data yang Disimpan di Database

Bab ini menjelaskan **semua data yang disimpan** oleh sistem: tabel apa saja yang ada, gunanya, arti tiap kolom, dan **dari mana** data itu berasal. Bahasanya dibuat sesederhana mungkin, jadi bisa dibaca tanpa latar belakang database.

10.1 Istilah dasar

Database bisa dibayangkan seperti **satu buku kerja Excel** berisi banyak *sheet*:

Istilah	Perumpamaan Excel	Penjelasan
Tabel	Satu sheet	Kumpulan data sejenis, misalnya tabel *users* berisi semua akun.
Baris	Satu baris di sheet	Satu data utuh, misalnya satu akun mahasiswa.
Kolom	Judul kolom (A, B, C…)	Satu jenis keterangan, misalnya kolom *email*.
id	Nomor baris	Nomor unik setiap baris, dibuat otomatis (1, 2, 3, …). Tidak pernah dipakai ulang.
…_id (foreign key)	Rujukan ke sheet lain	Kolom berakhiran *_id* berisi nomor id dari tabel lain. Contoh: *course_id = 5* artinya "milik mata kuliah nomor 5". Dengan begini data tidak perlu ditulis berulang.
created_at, **updated_at**	—	Waktu baris dibuat dan terakhir diubah. Diisi otomatis oleh Laravel dalam WIB. Ada di hampir semua tabel, jadi tidak diulang di daftar kolom di bawah.
Kosong (null)	Sel kosong	Kolom boleh tidak berisi, misalnya deskripsi yang tidak diisi.
Pilihan tetap (enum)	Dropdown	Kolom hanya boleh berisi nilai dari daftar tertentu, misalnya status: pending / processing / completed / failed.
JSON	Satu sel berisi catatan bertingkat	Beberapa data sekaligus dalam satu kolom, misalnya skor tiga lapis {dokumen, passage, kalimat}.
Cascade	Hapus berantai	Jika baris induk dihapus, baris yang merujuk kepadanya ikut terhapus. Contoh: hapus tugas → semua pengumpulan dan hasil analisisnya ikut terhapus.

10.2 Asal data

Setiap data di database berasal dari salah satu dari lima sumber ini. Kolom **"Asal data"** di tabel-tabel berikutnya memakai label yang sama:

Label	Artinya
Pengguna	Diketik atau dipilih orang di form: dosen, mahasiswa, atau peneliti.

Label	Artinya
Otomatis	Dibuat Laravel sendiri, misalnya nomor id, waktu, nama berkas acak, atau status.
Seeder	Diisi sekali saat instalasi, misalnya akun awal.
FastAPI	Hasil hitungan mesin analisis (embedding), dikirim lewat webhook.
AI	Jawaban Claude/Gemini, diteruskan FastAPI lewat webhook.

10.3 Peta hubungan antar tabel

Panah dibaca "satu … punya banyak …". Contoh: satu mata kuliah punya banyak tugas.

```
users
 ├─ dosen ──────> courses ──> assignments ──> submissions
 │                  │            │               │
 └─ mahasiswa ─> course_user      │               ├─> similarity_results
                                  │               │     (hasil per pasangan)
                                  │               ├─> ai_detection_results
                                  │               │     (hasil AI per dokumen)
                                  │               ├─> pair_labels  (validasi)
                                  │               └─> ai_labels    (validasi)
                                  └─> analysis_runs (riwayat analisis)

LAB PENGUJIAN (terpisah dari data kuliah):
experiment_datasets
 ├─> experiment_documents
 └─> experiment_runs
       ├─> experiment_pair_scores (per pasangan)
       └─> experiment_ai_scores   (per dokumen)
```

Catatan
Berkas fisik tidak disimpan di database. PDF/DOCX disimpan di folder *storage/app/public/submissions/* (tugas) dan *storage/app/public/experiments/* (Lab). Database hanya menyimpan **alamat** berkas itu (kolom *file_path*), seperti daftar isi yang menunjuk ke halaman buku.

10.4 Akun dan perkuliahan

Tabel users

Gunanya: Menyimpan semua akun yang bisa login: admin, dosen, mahasiswa, dan peneliti.

Diisi kapan: Saat instalasi (seeder) atau ketika akun ditambahkan.

Kolom	Artinya	Asal data	Contoh
name	Nama lengkap pengguna	Seeder	Andi Pratama
email	Email untuk login. Tidak boleh ada dua akun dengan email sama.	Seeder	andi@student…

Kolom	Artinya	Asal data	Contoh
password	Password yang sudah **diacak (hash)** dengan bcrypt. Password asli tidak pernah disimpan, sehingga admin pun tidak bisa melihatnya.	Seeder / Otomatis	$2y$12$Kq…
role	Peran pengguna, menentukan menu dan hak akses.	Seeder	mahasiswa
remember_token	Kode untuk fitur "ingat saya" saat login.	Otomatis	(acak)

Tabel courses

Gunanya: Daftar mata kuliah.

Diisi kapan: Saat dosen membuat mata kuliah.

Kolom	Artinya	Asal data	Contoh
dosen_id	Dosen pengampu (merujuk ke *users*). Hanya dosen ini yang boleh mengelola mata kuliahnya.	Otomatis (dosen yang login)	2
code	Kode mata kuliah, unik.	Pengguna	IF2201
name	Nama mata kuliah.	Pengguna	Basis Data
description	Keterangan tambahan, boleh kosong.	Pengguna	—

Tabel course_user

Gunanya: Daftar mahasiswa yang terdaftar di setiap mata kuliah. Satu baris = satu mahasiswa di satu mata kuliah.

Diisi kapan: Saat dosen mendaftarkan mahasiswa.

Kolom	Artinya	Asal data	Contoh
course_id	Mata kuliahnya.	Pengguna	5
user_id	Mahasiswanya.	Pengguna	7

> Tabel seperti ini disebut **tabel penghubung**: satu mahasiswa bisa ikut banyak mata kuliah, dan satu mata kuliah punya banyak mahasiswa.

Tabel assignments

Gunanya: Daftar tugas di setiap mata kuliah.

Diisi kapan: Saat dosen membuat atau mengubah tugas.

Kolom	Artinya	Asal data	Contoh
course_id	Tugas ini milik mata kuliah mana.	Otomatis	5
title	Judul tugas.	Pengguna	Tugas 2 – Normalisasi
description	Instruksi tugas, boleh kosong.	Pengguna	Kerjakan…

Kolom	Artinya	Asal data	Contoh
due_date	Tenggat pengumpulan (WIB). Setelah lewat, tugas otomatis tertutup.	Pengguna	2026-10-20 23:59
closed_at	Waktu dosen **menutup manual**. Kosong = tidak ditutup manual. Dikosongkan lagi bila dosen membuka kembali.	Otomatis (tombol tutup)	2026-10-18 10:02

Tabel submissions

Gunanya: Berkas tugas yang dikumpulkan mahasiswa. Satu mahasiswa hanya punya **satu** pengumpulan per tugas.

Diisi kapan: Saat mahasiswa mengunggah, mengganti, atau menghapus berkas.

Kolom	Artinya	Asal data	Contoh
assignment_id	Untuk tugas mana.	Otomatis	12
user_id	Mahasiswa pengumpul.	Otomatis (yang login)	7
file_path	Lokasi berkas di server, dengan nama acak (UUID) agar tidak bisa ditebak. Inilah yang dikirim ke FastAPI sebagai alamat unduhan.	Otomatis	submissions/12/9f1c….pdf
file_name	Nama asli berkas dari komputer mahasiswa, hanya untuk ditampilkan dan saat diunduh.	Pengguna (nama berkas)	Tugas2_Andi.pdf
created_at / updated_at	Waktu pertama mengumpulkan / terakhir mengganti berkas.	Otomatis	—

10.5 Hasil analisis kemiripan dan AI

Tabel analysis_runs

Gunanya: Riwayat setiap kali dosen menekan "Jalankan analisis". Satu baris = satu kali analisis. Berguna untuk melihat status, mengulang analisis, dan mencatat pengaturan yang dipakai agar hasilnya bisa dipertanggungjawabkan.

Diisi kapan: Dibuat saat tombol ditekan, lalu diperbarui saat FastAPI menerima pekerjaan dan saat hasil (webhook) datang.

Kolom	Artinya	Asal data	Contoh
assignment_id	Analisis untuk tugas mana.	Otomatis	12
triggered_by	Dosen yang menjalankan.	Otomatis	2
status	Tahap proses: *pending* (baru dibuat), *processing* (FastAPI sedang bekerja), *completed* (selesai), *failed* (gagal).	Otomatis	completed
threshold	Ambang skor gabungan yang dipilih dosen. Pasangan dengan skor ≥ ambang dianggap mencurigakan.	Pengguna	0.700
embedding_model	Nama model embedding yang dipakai.	FastAPI	paraphrase-multilingual-MiniLM-L12-v2

Kolom	Artinya	Asal data	Contoh
documents_sent	Jumlah berkas yang dikirim Laravel.	Otomatis	30
documents_processed	Jumlah berkas yang teksnya berhasil dibaca.	FastAPI	29
pairs_evaluated	Jumlah pasangan yang dibandingkan, yaitu N×(N−1)/2.	FastAPI	406
pairs_flagged	Jumlah pasangan di atas ambang.	FastAPI	6
metrics	Catatan teknis lengkap dalam JSON (lihat tabel di bawah).	FastAPI	{…}
error_message	Penyebab kegagalan, jika gagal.	Otomatis / FastAPI	Tidak ada respons…
started_at / finished_at	Waktu FastAPI mulai bekerja dan waktu hasil diterima.	Otomatis	10:02 / 10:05

Isi kolom metrics (JSON)

Kolom *metrics* berisi catatan yang dikirim FastAPI. Isinya sama untuk analisis dosen dan eksperimen Lab:

Kunci	Artinya
threshold, documents_received	Ambang yang dipakai dan jumlah berkas yang diterima FastAPI.
llm	Penyedia AI (anthropic/gemini), nama model, dan tingkat effort.
llm_usage	Jumlah panggilan AI, yang gagal, serta token masuk/keluar (dasar perhitungan biaya).
multilayer	Pengaturan Tahap 1 yang dipakai: bobot D/P/K, panjang passage, ambang kalimat, serta rata-rata jumlah kalimat dan passage per dokumen.
extraction_seconds, tier1_seconds, tier2_similarity_seconds, ai_detection_seconds, total_seconds	Lama waktu tiap tahap (detik).
failed_extractions	Daftar id pengumpulan yang teksnya tidak bisa dibaca.
document_checks	Untuk setiap dokumen: jumlah kata, halaman, gambar, dan peringatan "perlu diperiksa manual".
pairs_total, pairs_above_threshold	Jumlah semua pasangan dan yang melewati ambang.
llm_similarity_calls, llm_similarity_calls_avoided	Panggilan AI di Tahap 2 dan jumlah yang **dihemat** karena tersaring Tahap 1.
llm_similarity_errors, llm_ai_detection_calls, ai_detection_errors	Jumlah kegagalan AI dan jumlah dokumen yang dicek AI.

Tabel similarity_results

Gunanya: Hasil perbandingan setiap pasangan pengumpulan. Satu baris = satu pasangan (Andi–Siti, Andi–Budi, dst.). Dari tabel inilah daftar pasangan dan matriks di halaman hasil dibuat.

Diisi kapan: Saat webhook hasil analisis diterima. Hasil lama tugas yang sama dihapus dulu, lalu diganti yang baru.

Kolom	Artinya	Asal data	Contoh
assignment_id	Tugasnya.	Otomatis	12
submission_a_id, submission_b_id	Dua pengumpulan yang dibandingkan.	FastAPI	101, 104
similarity_score	**Skor gabungan** 0–1 = (0,2×D + 0,4×P + 0,4×K). Makin besar makin mirip.	FastAPI	0.8231
layer_scores	JSON skor tiap lapis: *document* (D, kemiripan topik), *passage* (P, kemiripan paragraf), *sentence* (K, persentase kalimat kembar).	FastAPI	{"document":0.91, …}
is_flagged	Ya/tidak: skor gabungan ≥ ambang. Hanya pasangan "ya" yang diperiksa AI.	FastAPI	1 (ya)
verdict	Kesimpulan singkat dari AI. Untuk pasangan di bawah ambang diisi "Di bawah ambang batas".	AI	Indikasi plagiarisme tinggi
action_recommendation	Saran tindakan: PERIKSA_MANUAL atau AMAN. Bila AI gagal, otomatis PERIKSA_MANUAL.	AI	PERIKSA_MANUAL
summary	Penjelasan AI tentang bukti kemiripan.	AI	Paragraf 2 dan 3…
matched_segments	JSON daftar bagian yang mirip. Setiap bagian berisi kutipan teks A, teks B, dan jenisnya: VERBATIM_COPY (salinan persis) atau PARAPHRASED (parafrase).	AI	[{…}, {…}]

Tabel ai_detection_results

Gunanya: Hasil deteksi tulisan AI untuk setiap pengumpulan. Satu baris = satu dokumen.

Diisi kapan: Saat webhook hasil analisis diterima (jika deteksi AI dijalankan).

Kolom	Artinya	Asal data	Contoh
assignment_id	Tugasnya.	Otomatis	12
submission_id	Pengumpulan yang diperiksa.	FastAPI	101
ai_probability	Perkiraan peluang ditulis AI, 0–1. Kosong jika gagal.	AI	0.1200
verdict	Kesimpulan dari probabilitas: LIKELY_AI (≥ 0,70), MIXED_AI (0,35–0,69), HUMAN_WRITTEN (< 0,35), atau ERROR (gagal dianalisis).	FastAPI (dari angka AI)	HUMAN_WRITTEN
analysis_summary	Alasan singkat dari AI.	AI	Variasi kalimat alami…
flagged_patterns	JSON daftar potongan kalimat yang dianggap khas AI.	AI	["Penting untuk dicatat…"]

10.6 Validasi dosen

Dua tabel ini berisi **penilaian dosen** yang dipakai sebagai kunci jawaban untuk mengukur akurasi sistem di halaman Evaluasi.

Tabel pair_labels

Gunanya: Penilaian dosen untuk sebuah pasangan: benar plagiat atau tidak.

Diisi kapan: Saat dosen menekan tombol validasi pada pasangan. Dihapus otomatis jika salah satu mahasiswa mengganti berkasnya.

Kolom	Artinya	Asal data	Contoh
assignment_id	Tugasnya.	Otomatis	12
submission_low_id, submission_high_id	Dua pengumpulan. Id yang lebih kecil selalu di *low*, supaya pasangan A–B dan B–A tidak tercatat dua kali.	Otomatis	101, 104
is_plagiarism	Keputusan dosen: plagiat (1) atau bukan (0).	Pengguna	1
note	Catatan dosen, boleh kosong.	Pengguna	—
labeled_by	Dosen yang menilai.	Otomatis	2

Tabel ai_labels

Gunanya: Penilaian dosen untuk satu dokumen: ditulis AI atau manusia.

Diisi kapan: Saat dosen menekan tombol validasi pada hasil deteksi AI.

Kolom	Artinya	Asal data	Contoh
assignment_id	Tugasnya.	Otomatis	12
submission_id	Pengumpulan yang dinilai (satu dokumen hanya punya satu label).	Pengguna	101
is_ai	Keputusan dosen: tulisan AI (1) atau manusia (0).	Pengguna	0
labeled_by	Dosen yang menilai.	Otomatis	2

10.7 Lab Pengujian

Tabel-tabel Lab terpisah dari data perkuliahan, sehingga percobaan tidak mengganggu nilai atau hasil mahasiswa.

Tabel experiment_datasets

Gunanya: Kumpulan dokumen uji (korpus) milik peneliti.

Diisi kapan: Saat peneliti membuat dataset atau korpus contoh.

Kolom	Artinya	Asal data	Contoh
name	Nama dataset.	Pengguna	Korpus esai 2026
description	Keterangan, boleh kosong.	Pengguna	—
is_sample	Ya jika ini korpus contoh bawaan (teks buatan AI, bukan data penelitian).	Otomatis	0

Kolom	Artinya	Asal data	Contoh
created_by	Peneliti pembuat.	Otomatis	9

Tabel experiment_documents

Gunanya: Dokumen di dalam dataset, **lengkap dengan labelnya**. Label inilah yang menjadi kunci jawaban otomatis.

Diisi kapan: Saat peneliti menambah dokumen satu per satu atau lewat impor massal.

Kolom	Artinya	Asal data	Contoh
dataset_id	Dataset pemiliknya.	Otomatis	3
title	Judul dokumen.	Pengguna	Esai 01 – parafrase
category	Jenis dokumen: ORIGINAL (asli), VERBATIM_COPY (salinan), PARAPHRASE_MANUAL, PARAPHRASE_AI, INDEPENDENT, AI_GENERATED.	Pengguna	PARAPHRASE_MANUAL
source_document_id	Dokumen sumber, untuk salinan/parafrase. Dipakai untuk menentukan "keluarga": satu keluarga = pasangan plagiat.	Pengguna	41
file_path	Lokasi berkas di server (teks yang ditempel juga disimpan sebagai .txt).	Otomatis	experiments/3/….txt
file_name	Nama asli berkas.	Pengguna	esai01_para.docx
word_count	Jumlah kata (hanya untuk teks/TXT; kosong untuk PDF/DOCX).	Otomatis	612

Tabel experiment_runs

Gunanya: Satu kali percobaan (eksperimen) pada sebuah dataset, beserta pengaturannya.

Diisi kapan: Dibuat saat peneliti menekan "Jalankan eksperimen", diperbarui saat hasil diterima.

Kolom	Artinya	Asal data	Contoh
dataset_id	Dataset yang diuji.	Otomatis	3
created_by	Peneliti yang menjalankan.	Otomatis	9
name	Nama percobaan agar mudah dibandingkan.	Pengguna	Ambang kalimat 0,80
status	pending / processing / completed / failed.	Otomatis	completed
config	JSON pengaturan: ambang, bobot D/P/K, panjang passage, ambang kalimat, Tahap 2 ya/tidak, deteksi AI ya/tidak.	Pengguna	{"threshold":0.7, …}
metrics	Catatan teknis dari FastAPI, isinya sama seperti *analysis_runs.metrics*.	FastAPI	{…}
error_message, started_at, finished_at	Sama seperti di *analysis_runs*.	Otomatis	—

Tabel experiment_pair_scores

Gunanya: Skor **mentah** setiap pasangan dokumen dalam satu eksperimen. Karena D, P, K disimpan terpisah, bobot dan ambang lain bisa dihitung ulang tanpa menjalankan FastAPI lagi.

Diisi kapan: Saat webhook hasil eksperimen diterima.

Kolom	Artinya	Asal data	Contoh
run_id	Eksperimennya.	Otomatis	5
document_low_id, document_high_id	Dua dokumen (id kecil di *low*).	FastAPI	41, 44
document_score	Skor D, kemiripan topik.	FastAPI	0.9120
passage_score	Skor P, kemiripan paragraf.	FastAPI	0.8410
sentence_score	Skor K, persentase kalimat kembar.	FastAPI	0.6500
combined_score	Skor gabungan dengan bobot eksperimen ini.	FastAPI	0.7978
llm_checked	Ya jika pasangan ini diperiksa AI (Tahap 2).	FastAPI	1
llm_action	Rekomendasi AI: PERIKSA_MANUAL atau AMAN.	AI	PERIKSA_MANUAL
llm_error	Ya jika panggilan AI gagal.	FastAPI	0
llm_result	JSON ringkas: kesimpulan, ringkasan, dan jumlah segmen salinan/parafrase.	AI	{…}

Label plagiat **tidak disimpan** di tabel ini. Label dihitung saat halaman dibuka, dari kategori dan sumber di *experiment_documents*, sehingga perbaikan label dokumen langsung berlaku untuk semua eksperimen.

Tabel experiment_ai_scores

Gunanya: Hasil deteksi AI setiap dokumen dalam satu eksperimen.

Diisi kapan: Saat webhook hasil eksperimen diterima (jika deteksi AI dijalankan).

Kolom	Artinya	Asal data	Contoh
run_id	Eksperimennya.	Otomatis	5
document_id	Dokumen yang diperiksa.	FastAPI	41
ai_probability	Peluang ditulis AI, 0–1. Dipakai untuk mencoba berbagai ambang AI.	AI	0.8500
verdict	LIKELY_AI / MIXED_AI / HUMAN_WRITTEN / ERROR.	FastAPI	LIKELY_AI
analysis_summary	Alasan singkat dari AI.	AI	Struktur paragraf seragam…

10.8 Tabel bawaan Laravel

Tabel	Gunanya	Dipakai?
migrations	Catatan migration mana yang sudah dijalankan, supaya struktur tabel tidak dibuat dua kali.	Ya, otomatis oleh Laravel
sessions	Tempat menyimpan sesi login jika sesi disimpan di database.	Tidak. Sesi disimpan di berkas (SESSION_DRIVER=file).
personal_access_tokens	Token untuk login lewat API (Laravel Sanctum).	Tidak. Sistem ini belum memakai login API.

10.9 Contoh perjalanan data

Supaya lebih terbayang, berikut data yang tercipta dari awal sampai akhir untuk satu tugas dengan 3 mahasiswa:

1. Dosen membuat tugas → **1 baris** di *assignments*.
2. Andi, Siti, dan Budi mengunggah → **3 baris** di *submissions* + 3 berkas di folder storage.
3. Dosen menekan "Jalankan analisis" → **1 baris** di *analysis_runs* (status *pending* → *processing*).
4. FastAPI membandingkan 3 pasangan (Andi–Siti, Andi–Budi, Siti–Budi) → **3 baris** di *similarity_results*.
5. FastAPI memeriksa tulisan AI di 3 dokumen → **3 baris** di *ai_detection_results*.
6. Baris di *analysis_runs* diperbarui: status *completed*, jumlah pasangan, dan *metrics*.
7. Dosen menilai pasangan Andi–Siti sebagai plagiat → **1 baris** di *pair_labels*.
8. Jika Siti mengganti berkasnya, hasil analisis dan label yang melibatkan Siti dihapus karena isinya sudah berbeda. Dosen perlu menjalankan analisis lagi.

Jika dosen menjalankan analisis kedua kalinya, baris baru dibuat di *analysis_runs* (riwayat tetap ada), tetapi isi *similarity_results* dan *ai_detection_results* tugas itu **diganti** dengan hasil terbaru.

11. Kalau Ingin Mengubah Sesuatu

Tabel ini membantu menemukan file yang perlu dibuka untuk perubahan yang sering diperlukan.

Ingin mengubah...	Buka file
Ambang bawaan dan batas waktu analisis	`laravel-app/config/services.php` atau .env
Bobot, panjang passage, ambang kalimat bawaan	`docker-compose.yml` (environment fastapi)
Cara memecah kalimat/passage atau rumus D/P/K	`fastapi-service/multilayer.py`
Isi perintah (prompt) ke AI	`fastapi-service/main.py` — analyze_similarity_with_llm(), detect_ai_generated_text()
Model AI yang dipakai	.env: CLAUDE_MODEL, LLM_EFFORT; logikanya di `llm.py`

Ingin mengubah…	Buka file
Aturan upload mahasiswa (format, ukuran)	`app/Http/Requests/Mahasiswa/SubmissionFileRequest.php`
Siapa boleh melakukan apa	`app/Policies/` dan middleware di `routes/web.php`
Tampilan halaman	`resources/views/` (lalu jalankan *npm run build*)
Rumus evaluasi (precision, recall, F1)	`app/Services/EvaluationService.php` — confusion()
Struktur tabel database	`database/migrations/` (buat migration baru, jangan ubah yang lama), lalu *php artisan migrate*
Perhitungan di Lab	`app/Services/ExperimentEvaluator.php`

Kebiasaan yang membantu

Setiap selesai mengubah kode, jalankan test: *docker exec campus_laravel php artisan test*.
Kalau analisis tidak kunjung selesai, lihat log FastAPI: *docker compose logs -f fastapi*. Pesan seperti *[TASK FAILED]* atau *[CALLBACK FAILED]* menunjukkan di mana masalahnya.
Kalau mengubah .env root, jalankan *docker compose up -d* agar container membaca nilai baru.